```python
import random
import itertools
import math
import matplotlib.pyplot as plt
from typing import List, Tuple, Dict


class Dice:
    """
    Represents a single die.

    A Dice object can be created using either:
    - An integer representing the number of faces (labels 1 to n), or
    - A list of custom face labels.

    Attributes:
        face_count (int): Number of faces on the die.
        face_labels (List[int]): Labels representing each face.
    """

    def __init__(self, faces):
        """
        Initialises a Dice instance.

        Args:
            faces (int | List[int]):
                If int, creates a standard die numbered 1 to faces.
                If list, creates a die with custom face labels.

        Raises:
            TypeError: If faces is not an int or a list.
        """
        if isinstance(faces, int):
            self.face_count = faces
            self.face_labels = list(range(1, faces + 1))
        elif isinstance(faces, list):
            self.face_count = len(faces)
            self.face_labels = faces
        else:
            raise TypeError("faces must be an int or a list of ints")

    def roll(self) -> int:
        """
        Rolls the die once.

        Returns:
            int: The randomly selected face value.
        """
        return self.face_labels[random.randint(0, self.face_count - 1)]

    def roll_n(self, times: int) -> List[int]:
        """
        Rolls the die multiple times.

        Args:
            times (int): The number of times to roll the die.

        Returns:
            List[int]: A list containing the results of each roll.
        """
```

```
61          return [self.roll() for _ in range(times)]
62
63
64  class DiceRoll:
65      """
66      Represents a combined roll of multiple dice with an optional modifier.
67
68      Attributes:
69          dice (List[Dice]): List of Dice objects.
70          count (List[int]): Number of times each die is rolled.
71          modifier (int): Value added to the final total.
72      """
73
74      def __init__(self, roll: List[Tuple[Dice, int]], modifier: int = 0):
75          """
76          Initialises a DiceRoll instance.
77
78          Args:
79              roll (List[Tuple[Dice, int]]):
80                  A list of tuples where each tuple contains:
81                  (Dice object, number of times it should be rolled).
82              modifier (int, optional): A value added to the total result.
83          """
84          self.dice = [pair[0] for pair in roll]
85          self.count = [pair[1] for pair in roll]
86          self.modifier = modifier
87
88      def roll(self) -> Tuple[int, List[int]]:
89          """
90          Performs the dice roll operation.
91
92          Rolls each die the specified number of times and adds
93          the modifier to the total.
94
95          Returns:
96              Tuple[int, List[int]]:
97                  The total result including modifier,
98                  and a list of individual roll results.
99          """
100         results = []
101
102         for die, count in zip(self.dice, self.count):
103             results.extend(die.roll_n(count))
104
105         total = sum(results) + self.modifier
106         return total, results
107
108     def possible_outcomes(self) -> List[Tuple[int, List[int]]]:
109         """
110         Generates all possible outcome combinations.
111
112         Uses Cartesian product to compute all permutations
113         of face values for the configured dice.
114
115         Returns:
116             List[Tuple[int, List[int]]]:
117                 A list of tuples containing:
118                 (total including modifier, list of face values).
119         """
120         face_lists = []
```

```
121
122         for die, count in zip(self.dice, self.count):
123             face_lists.extend([die.face_labels] * count)
124
125         permutations = itertools.product(*face_lists)
126
127         outcomes = []
128         for perm in permutations:
129             roll_list = list(perm)
130             total = sum(roll_list) + self.modifier
131             outcomes.append((total, roll_list))
132
133         return outcomes
134
135
136 class RollAnalyser:
137     """
138     Performs statistical analysis on a DiceRoll object.
139
140     Attributes:
141         dice_roll (DiceRoll): The dice configuration being analysed.
142         outcomes (List[Tuple[int, List[int]]]): All possible outcomes.
143     """
144
145     def __init__(self, dice_roll: DiceRoll):
146         """
147         Initialises the RollAnalyser.
148
149         Args:
150             dice_roll (DiceRoll): The DiceRoll instance to analyse.
151         """
152         self.dice_roll = dice_roll
153         self.outcomes = dice_roll.possible_outcomes()
154
155     def outcome_frequencies(self) -> Dict[int, int]:
156         """
157         Calculates the frequency of each possible total.
158
159         Returns:
160             Dict[int, int]: A dictionary mapping totals to frequencies.
161         """
162         frequencies = {}
163
164         for total, _ in self.outcomes:
165             frequencies[total] = frequencies.get(total, 0) + 1
166
167         return frequencies
168
169     def average(self) -> float:
170         """
171         Calculates the mean (expected value) of all possible outcomes.
172
173         Returns:
174             float: The average total.
175         """
176         totals = [total for total, _ in self.outcomes]
177         return sum(totals) / len(totals)
178
179     def standard_deviation(self) -> float:
180         """
```

```
        Calculates the standard deviation of all possible totals.

        Returns:
            float: The standard deviation.
        """
        avg = self.average()
        totals = [total for total, _ in self.outcomes]
        variance = sum((x - avg) ** 2 for x in totals) / len(totals)
        return math.sqrt(variance)

    def plot_histogram(self) -> None:
        """
        Displays a histogram showing the distribution
        of possible roll outcomes.
        """
        frequencies = self.outcome_frequencies()

        plt.bar(frequencies.keys(), frequencies.values())
        plt.xlabel("Roll Outcome")
        plt.ylabel("Frequency")
        plt.title("Distribution of Possible Dice Roll Outcomes")
        plt.show()


def run_tests() -> None:
    """
    Runs a demonstration of the Dice simulation system.

    This function creates sample dice, performs rolls,
    analyses results, and displays a histogram.
    """
    print("Running test script...\n")

    # Create dice
    d4 = Dice(4)
    d6 = Dice(6)
    d6_unlucky = Dice([1, 1, 1, 1, 1, 6])

    # Test individual dice
    print(f"1d4 roll: {d4.roll()}")
    print(f"20d4 rolls: {d4.roll_n(20)}\n")

    print(f"1d6 roll: {d6.roll()}")
    print(f"20d6 rolls: {d6.roll_n(20)}\n")

    print(f"1d6 unlucky roll: {d6_unlucky.roll()}")
    print(f"20d6 unlucky rolls: {d6_unlucky.roll_n(20)}\n")

    print(f"Max d6 (1000 rolls): {max(d6.roll_n(1000))}")
    print(f"Min d6 (1000 rolls): {min(d6.roll_n(1000))}")
    print(f"Average d6 (100000 rolls): {sum(d6.roll_n(100000)) / 100000}\n")

    # Complex roll example
    print("Creating a complex roll: 4d4 + 1d20 - 2")
    complex_roll = DiceRoll([(d4, 4), (Dice(20), 1)], -2)
    total, rolls = complex_roll.roll()
    print(f"Result: {total} from rolls {rolls}\n")

    # Statistical analysis
    analyser = RollAnalyser(DiceRoll([(Dice(4), 4)]))
```

```
241     print("Average outcome (4d4):", analyser.average())
242     print("Standard deviation (4d4):", analyser.standard_deviation())
243 
244     analyser.plot_histogram()
245 
246 
247 if __name__ == "__main__":
248     """
249     Entry point of the program.
250     """
251     run_tests()
```